CSE 318 Assignment-03: Chain Reaction Game Report

Student ID: 2105114

June 16, 2025

1 Introduction

This report evaluates the Chain Reaction game implemented for CSE 318 Assignment-03. The game, played on a 6x9 grid, involves two players (Red and Blue) placing orbs to trigger chain reactions, aiming to eliminate the opponent's orbs or cause an infinite explosion. We assess five domain-inspired heuristics (`h1` to `h5`) and a combined heuristic (`evaluate_combined`) in three modes: Human vs. AI, AI vs. AI, and Random Agent vs. AI. The report details the heuristic rationales, experimental setup, results, and discusses performance and trade-offs.

2 Heuristic Rationales

The following heuristics, implemented in `minimax.py`, are designed to capture strategic aspects of Chain Reaction:

- **h1: Orb Count Difference** Computes Blue orbs minus Red orbs. *Rationale*: More orbs grant numerical advantage, aligning with the goal of eliminating opponent orbs. It's effective early-game but ignores positional strategy.
- **h2: Critical Mass Proximity** Sums orb-to-critical-mass ratios for Blue (positive) and Red (negative). *Rationale*: Cells near critical mass trigger chain reactions, supporting mid-game explosion strategies and infinite explosion wins.
- **h3: Board Control** Counts Blue-controlled cells minus Red-controlled cells. *Rationale*: Controlling more cells limits opponent moves, crucial for late-game territorial dominance.
- **h4: Explosion Potential** Scores Blue cells near critical mass with Red neighbors (positive), penalizes Red cells near Blue (negative). *Rationale*: Prioritizes moves capturing opponent cells via chain reactions, key for mid-game and infinite explosions.
- **h5: Stability and Vulnerability** Rewards Blue stability (low orb-to-critical-mass ratio) and Red vulnerability near Blue. *Rationale*: Balances defense (stable Blue cells) and offense (exploiting Red), vital for late-game and avoiding self-triggered explosions.

- **evaluate_combined** Weighted sum of `h1` to `h5` (weights: 1.0, 1.0, 1.0, 2.0, 2.0). *Rationale*: Combines all aspects for robust evaluation across game phases, with emphasis on `h4` and `h5` for dynamic and defensive play.

3 Experimental Setup

Experiments evaluated the heuristics in three matchups:

- **Matchups**:
  - `h1` vs. `evaluate_combined` (H1 vs. Combined, Red: H1, Blue: Combined)
  - Random Agent vs. `evaluate_combined` (Random vs. Combined, Red: Random, Blue: Combined)
  - `h4` vs. `h4` (H4 vs. H4, Red: H4, Blue: H4)
- **Depths**:
  - AI1 (Red): Depth = 4 (AI vs. AI modes).
  - AI2 (Blue): Depth = 2 (all modes).
  - Random Agent: Random valid moves (no depth).
- **Games**: 10 per matchup for statistical reliability.
- **Win Conditions**:
  - **Standard Win**: One player retains orbs while the other has none.
  - **Infinite Explosion Win**: Player triggering a cycle (same board state and exploding cells) wins.

The experiment used a Python script (`experiments.py`) to track metrics, ensuring consistent evaluation.

4 Results

Table 1 summarizes Blue's win rates and game statistics for the three matchups.

Table 1: Experimental Results for Blue (Player 2)

Matchup	**Standard Wins (%)**	**Infinite Wins (%)**
H1 vs. Combined	70	0
Random vs. Combined	92	3
H4 vs. H4	50	0

5 Discussion

`evaluate_combined` outperformed `h1` with a 70% standard win rate in H1 vs. Combined, leveraging its comprehensive evaluation (Section 2) to balance orb count, chain reactions, and stability (Table 1). Key points:

- **Heuristic Performance**:
  - `evaluate_combined` excelled against Random Agent (92% standard wins) due to its integration of `h4` (explosion potential) and `h5` (stability), exploiting non-strategic moves.
  - Against `h1`, `evaluate_combined` won 70% of games, as `h1`'s orb count focus lacked positional and chain reaction strategies.
  - `h4` vs. `h4` resulted in a balanced 50% win rate, reflecting identical strategies and highlighting `h4`'s focus on explosion potential.
- **Infinite Explosion Wins**: Rare (0–3%), mostly in Random vs. Combined, confirming robust cycle detection in `engine.py`. Random moves occasionally trigger cycles, unlike AI's strategic play.
- **Trade-offs**:
  - `evaluate_combined` increased computation time due to its complexity, a trade-off for higher win rates (70% and 92% standard wins).
  - Increasing alpha in alpha-beta pruning enhances agent performance by tightening search bounds, leading to better move selection, but significantly slows computation due to deeper exploration.
  - Depth=4 for AI1 (Red) improved Red's performance, particularly in H1 vs. Combined and H4 vs. H4
  - Random Agent games were faster (in Random vs. Combined) but less competitive, as non-strategic moves led to high losses (92% standard wins for Blue).
  - Heuristics `h4` and `h5` outperformed `h1`, `h2`, and `h3` due to their focus on explosion potential and stability/vulnerability, respectively, as evidenced by the balanced 50% win rate in H4 vs. H4 and `evaluate_combined`'s reliance on their higher weights.

6 Conclusion

`evaluate_combined` was the most effective heuristic, achieving high win rates against `h1` and Random Agent by leveraging all game aspects. Its computational cost suggests a trade-off with efficiency. The infinite explosion rule added strategic depth, though rare in AI vs. AI. Future work could optimize heuristic weights or depths for faster play.